

Modelo-Vista- Controlador

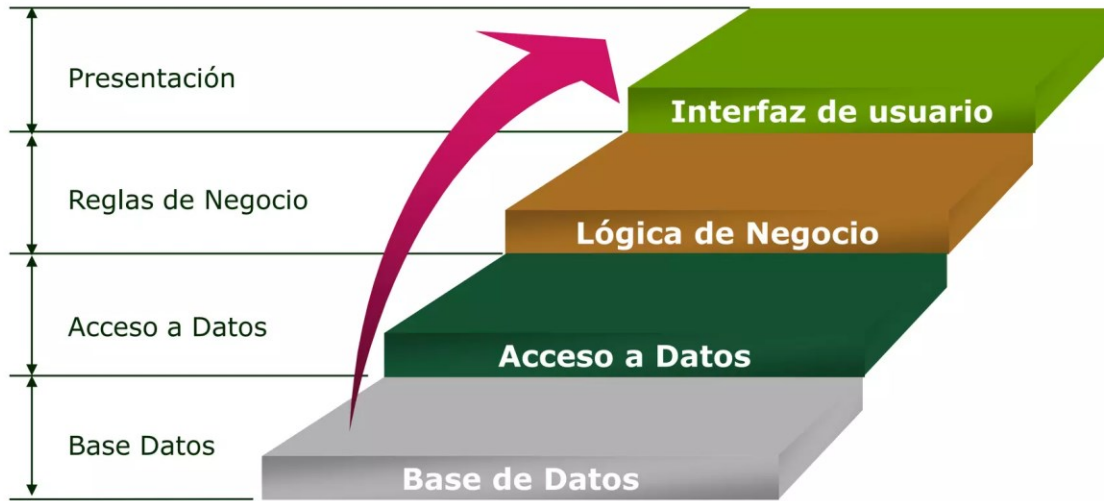
MVC



Desarrollo del software

- + En el desarrollo de software, la arquitectura en capas es un modelo de diseño que se utiliza para separar las diferentes funcionalidades del sistema en capas o niveles, donde cada capa se encarga de un conjunto de tareas específicas y se comunica con los niveles adyacentes mediante interfaces bien definidas
- + El desarrollo de software suele dividirse en tres capas fundamentales:
 - La lógica de presentación
 - La lógica de negocio
 - La lógica de datos
- + Cada una de estas capas cumple un propósito específico en la aplicación y se encarga de manejar diferentes aspectos del sistema.

Modelo de capas



Desarrollo del software

Desarrollo del software

Capa de negocio

- + La lógica de negocio es el núcleo del software y se encarga de representar las reglas y procesos que rigen la funcionalidad principal de la aplicación.
- + Esta capa determina cómo se realiza el procesamiento de datos y las acciones que pueden llevarse a cabo en el sistema.
- + Es el "corazón" de la funcionalidad del sistema, y gestiona cómo deben procesarse los datos que recibe de la capa de presentación, realizando validaciones, cálculos, y otros procesos necesarios antes de interactuar con la capa de datos.
- + Se definen las reglas de negocio, validaciones de usuarios, verificación de datos, cálculos y procesos que son esenciales para el funcionamiento de la aplicación.
- + Ejemplo: Si un usuario compra un producto, la capa de negocio valida que el inventario esté disponible, calcula el precio total con impuestos, y luego envía la solicitud de almacenamiento de la compra a la capa de datos.

Desarrollo del software

Capa de acceso a datos

- + La lógica de datos se encarga de la **gestión y persistencia de la información** que utiliza la aplicación.
- + Esto implica el almacenamiento, recuperación y manipulación de datos, generalmente en una base de datos. Su función es realizar operaciones de acceso, modificación, almacenamiento y eliminación de datos (CRUD), asegurándose de que los datos estén disponibles para la capa de negocio.
- + Aquí se definen las estructuras de datos, las relaciones entre las entidades, las consultas a la base de datos y la sincronización de datos entre la aplicación y el almacenamiento de datos.
- + Las tecnologías comunes en esta capa incluyen sistemas de gestión de bases de datos (como MySQL, PostgreSQL, MongoDB) y lenguajes de consulta (SQL).

Desarrollo del software

Capa de presentación

- + La lógica de presentación, también llamada **capa de usuario** se encarga de la interfaz de usuario y cómo los usuarios interactúan con el software. Esta capa controla la presentación de la información en la pantalla, la navegación y la interacción con el usuario.
- + Aquí se definen elementos como la estructura de las páginas, formularios, botones y cómo se muestran los datos al usuario.
- + Las tecnologías utilizadas en esta capa pueden incluir HTML, CSS, JavaScript y frameworks de desarrollo de interfaz de usuario.
- + Ejemplo: Cuando un usuario completa un formulario en una página web y envía sus datos, es la capa de presentación la que se encarga de recolectar y enviar esta información a la capa de negocio, o de mostrar un mensaje si hubo algún error.

Model View Controller (MVC)

- + El proceso de desarrollo de software llamado Model View Controller (MVC), que **es un estándar** de programación de aplicaciones, utilizado tanto para hacer sitios web como programas tradicionales.
- + MVC (Model-View-Controller) es un patrón de diseño de software relacionado con la interconexión de los tres tipos de componentes principales en un lenguaje de programación como PHP y a menudo con un fuerte enfoque en la programación orientada a objetos (POO).
- + El patrón MVC se utiliza en muchos frameworks y entornos de desarrollo modernos, como Laravel en PHP, Django en Python y Spring MVC en Java, ya que ayuda a crear aplicaciones organizadas y eficientes.
- + Estos tres tipos de componentes son llamados modelos, vistas y controladores.

La Vista

- + **La Vista** es donde se encontrarán todos los elementos de la interfaz de usuario de una aplicación, esta puede contener código HTML, hojas de estilo CSS y archivos Javascript.
- + Cualquier cosa que el usuario pueda ver, se guarda en la vista.
- + Una Vista en MVC puede ser una página web completa o bien fragmentos de una página que se combinan para formar una presentación final. En muchas aplicaciones, lo que el usuario ve en un momento dado es el resultado de múltiples vistas que se ensamblan en una sola respuesta. Por ejemplo, una página web puede incluir un encabezado, un pie de página, un menú de navegación y una sección de contenido principal, cada uno de los cuales podría estar representado por una vista independiente.
- + La Vista no debe contener lógica de negocio; simplemente recibe los datos procesados por el Modelo a través del Controlador y los muestra.
- + Ejemplo: Siguiendo con el ejemplo de la tienda en línea, una Vista mostraría la lista de productos, el formulario de registro de usuario o la página de confirmación de compra.

El Modelo

- + **En el Modelo** mantendremos encapsulada la complejidad de nuestra base de datos y crearemos funciones para recibir, insertar, actualizar o borrar información de nuestras tablas. (CRUD). "CRUD" es un acrónimo que proviene de las palabras en inglés "**C**reate" (Crear), "**R**ead" (Leer), "**U**update" (Actualizar) y "**D**delete" (Borrar)
- + La implementación de operaciones CRUD simplifica el proceso de desarrollo y mantenimiento del software, ya que los desarrolladores en el controlador pueden centrarse en implementar la lógica de negocios de la aplicación, ya que la lógica de datos está encapsulada en el modelo.
- + Al mantenerse todas las llamadas a la base de datos en un mismo código, desde otras partes del programa podremos invocar las funciones que necesitemos del modelo y éste se encargará de procesarlas.
- + En el modelo nos podrán preocupar cosas como el tipo de base de datos con la que trabajamos, o las tablas y sus relaciones, **pero desde las otras partes del programa simplemente llamaremos a los métodos del modelo** sin importarnos qué tiene que hacer éste para realizar las acciones invocadas.
- + **El modelo representa las estructuras de datos, normalmente, en clases que contendrán métodos** que se ocuparán de recuperar, insertar y actualizar información en la base de datos.
- + En una aplicación de tienda en línea, el Modelo gestionaría la información de productos, clientes, pedidos, inventario, etc. Por ejemplo, cuando un cliente hace una compra, el Modelo actualiza la cantidad de productos en el inventario.

El Controlador

- + **El Controlador** es el componente encargado de conectar el Modelo con la Vista.
- + El Controlador en MVC actúa como un intermediario entre la Vista y el Modelo.
- + Gestiona las solicitudes del usuario, procesa la entrada del usuario y realiza acciones en consecuencia, como la **actualización del modelo o la redirección a una vista** específica.
- + Los controladores son el primer punto de entrada en estos componentes, ya que la primera solicitud se pasa a un Controlador.
- + El Controlador recibe las entradas del usuario (como clics, formularios, URL, etc.), las interpreta, y decide qué Modelo debe procesar esa solicitud. Una vez que el Modelo realiza las operaciones necesarias, el Controlador elige la Vista adecuada para mostrar los resultados al usuario.

Validaciones: ¿Controlador o Modelo?

El **Controlador** y el **Modelo**, tienen roles importantes en la validación, pero en distintos niveles y tipos de validación:

1. Validación en el Controlador:

- a. El **Controlador** es el primer lugar donde los datos introducidos por el usuario llegan desde la Vista. Su rol es hacer una validación inicial para asegurarse de que los datos son **completos** y **correctamente formateados** antes de pasarlos al Modelo.
- b. **Tipo de validación:** Validaciones de tipo **superficial o "rápidas"**. Por ejemplo, si se espera un campo de texto, asegurarse de que el campo no esté vacío; si se espera un correo electrónico, comprobar que esté en el formato correcto; y si se espera un número, verificar que tenga el tipo adecuado.

2. Validación en el Modelo:

- a. El **Modelo** realiza una **validación profunda**, asegurándose de que los datos cumplen con las reglas de negocio y restricciones propias de la lógica de la aplicación. Esta validación es necesaria porque el Modelo trabaja directamente con los datos que serán almacenados y manipulados, por lo que debe garantizar que cumplan con los requisitos del sistema.
- b. **Tipo de validación:** Validaciones de negocio, integridad y consistencia. Por ejemplo, verificar que un email no esté duplicado antes de crear un usuario.

Validaciones: ¿Controlador o Modelo?

1. Validación en el Controlador:

- A. **Objetivo:** Validar que los datos estén presentes, tengan el tipo de dato correcto y que respeten un formato general.
- B. **Ejemplo:** En una aplicación de registro de usuarios, el Controlador podría verificar que:
 - El campo de email esté presente y tenga el formato de un correo electrónico.
 - El campo de contraseña cumpla con una longitud mínima.

2. Validación en el Modelo:

- A. **Objetivo:** Verificar que los datos cumplan con restricciones de negocio, como unicidad, integridad, y coherencia.
- B. **Ejemplo:** Usando el mismo caso de registro de usuarios, el Modelo podría validar:
 - Que el email no esté duplicado en la base de datos.
 - Que la contraseña cumpla con ciertos criterios de seguridad específicos.

Validaciones: ¿Controlador o Modelo?

Validación en el Controlador

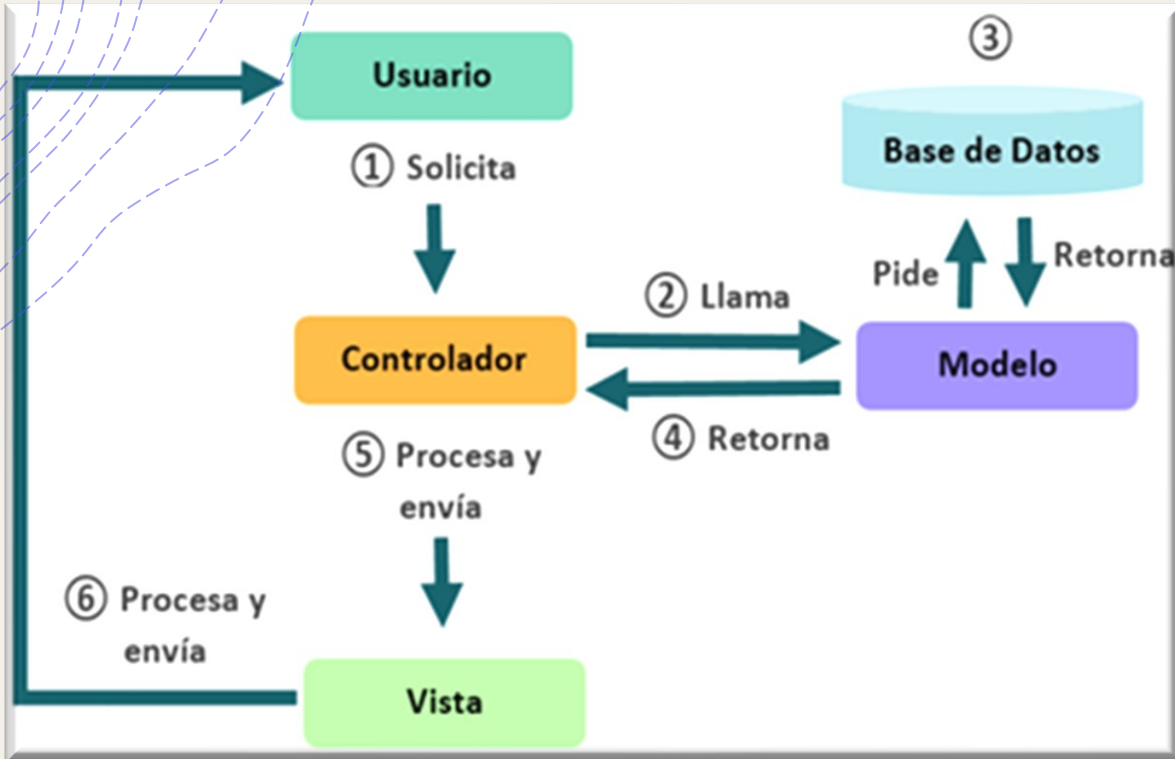
El Controlador realiza una validación superficial, asegurándose de que los datos necesarios estén presentes y en el formato correcto.

- A. Objetivo: Validar que los datos básicos de un producto están completos y siguen un formato general.
- B. Ejemplo:
 - Verificar que los campos como nombre, precio, y stock están presentes.
 - Comprobar que precio y stock son numéricos y tienen valores positivos.

Validación en el Modelo:

- A. Objetivo: Validar que los datos cumplen con las reglas de negocio, como que el código del producto sea único y que el stock inicial y el precio sean consistentes con la política de la empresa.
- B. Ejemplo:
 - Verificar que el código del producto no esté duplicado en la base de datos.
 - Comprobar que el precio esté dentro de un rango lógico (por ejemplo, mayor que el costo de producción).
 - Asegurarse de que el stock inicial no exceda la capacidad de inventario.

Ciclo de vida del MVC



1. **Usuario:** Interactúa con la Vista, ya sea haciendo clic, enviando un formulario o cualquier otra acción.
2. **Vista al Controlador:** La Vista envía la acción al Controlador para que se procese la solicitud.
3. **Controlador al Modelo:** El Controlador solicita al Modelo los datos necesarios, o le pide que realice una operación (como guardar un cambio).
4. **Modelo al Controlador:** El Modelo realiza la operación solicitada (consulta, almacenamiento o procesamiento de datos) y devuelve los datos o resultados al Controlador.
5. **Controlador a la Vista:** El Controlador pasa los datos que recibió del Modelo a la Vista.
6. **Vista al Usuario:** Finalmente, la Vista muestra la información actualizada o los resultados de la operación al usuario.

Ventajas y desventajas del MVC

Ventajas

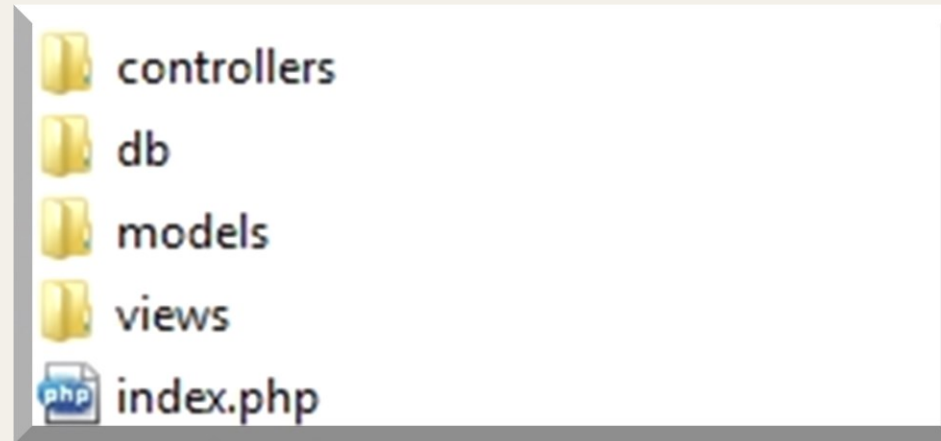
1. La separación del *Modelo* y la *Vista*, lo cual logra separar los datos, de su representación visual.
2. Facilita el manejo de errores.
3. Permite que el sistema sea escalable (aumentar sus componentes o funciones sin comprometer la operabilidad y la calidad).

Desventajas

1. La cantidad de archivos que se deben mantener incrementa considerablemente.
2. La curva de aprendizaje es más alta que utilizando otros modelos.
3. Su separación en capas aumenta la complejidad del sistema.

¿Como implementar el MVC en PHP?

- + Para implementar el MVC es imprescindible crear una estructura de ficheros parecida a esta:



Controlador Frontal

- + Debemos crear un controlador frontal, que se encargue de llamar a los diferentes controladores en función de la información que llegue a través de la URL.

```

class FrontController
{
    public function cargarControlador($nombreControlador)
    {
        switch ($nombreControlador) {
            case "producto":
                require_once "controllers/ProductoController.php";
                $controlador = new ProductoController();
                break;
            case "usuario":
                require_once "controllers/UsuarioController.php";
                $controlador = new UsuarioController();
                break;
            case "pedido":
                require_once "controllers/PedidoController.php";
                $controlador = new PedidoController();
                break;
            default:
                $this->cargarControladorDefecto();
                return;
        }
        // Llamar a un método por defecto o uno especificado en la URL
        if (isset($_GET["action"]) && method_exists($controlador, $_GET["action"])) {
            $accion = $_GET["action"];
            $controlador->$accion();
        } else {
            $controlador->index(); // Método por defecto
        }
    }
    private function cargarControladorDefecto()
    {
        require_once "controllers/HomeController.php";
        $controlador = new HomeController();
        $controlador->index();
    }
    public function manejarPetición()
    {
        if (isset($_GET["controller"])) {
            $this->cargarControlador($_GET["controller"]);
        } else {
            $this->cargarControladorDefecto();
        }
    }
}
// Crear una instancia del FrontController y manejar la petición
$frontController = new FrontController();
$frontController->manejarPetición();

```

Ejemplos de URL

Cargar la página de inicio (Controlador por defecto)

URL: index.php

Descripción: Carga el controlador por defecto (HomeController) y ejecuta el método `index()`, mostrando la página de inicio.

Ver la lista de productos

URL: index.php?controller=producto

Descripción: Carga `ProductoController` y ejecuta el método `index()`, mostrando la lista de productos.

Agregar un nuevo producto

URL: index.php?controller=producto&action=agregar

Descripción: Carga `ProductoController` y ejecuta el método `agregar()`, mostrando un formulario para agregar un nuevo producto.

Ver la lista de usuarios

URL: index.php?controller=usuario

Descripción: Carga `UsuarioController` y ejecuta el método `index()`, mostrando la lista de usuarios.

Editar un usuario específico

URL: index.php?controller=usuario&action=editar

Descripción: Carga `UsuarioController` y ejecuta el método `editar()`, mostrando un formulario para editar un usuario específico.